WEB TECHNOLOGIES

Unit 4: Advance Scripting, DHTML, AJAX, XML & Web Services

Rgpvnotes.tech - Your Academic Excellence Partner

UNIT 4: Advance Scripting | JavaScript Objects | DOM | Forms & Validations | DHTML & Events | AJAX | XML, XSL, XSLT, DTD & Schemas | Web Services

Table of Contents

Topic 1: JavaScript Objects & Built-in Objects

Topic 2: DHTML, DOM, and Browser Environments

Topic 3: Forms and Validations

Topic 4: AJAX (Asynchronous JavaScript and XML)

Topic 5: XML Basics, Key Components, DTD & Schemas

Topic 6: XSL, XSLT and XML Applications

Topic 7: Introduction to Web Services

Revision Notes

Important Questions

1 JavaScript Objects & Built-in Objects

Definition:

In JavaScript, an **Object** is a standalone entity, with properties and type. It is a collection of key-value pairs where keys are strings (property names) and values can be any data type, including functions (methods). JavaScript is highly object-based, and it provides several **Built-in Objects** to perform standard tasks.

Detailed Explanation:

Unlike primitive data types (like numbers or strings) that hold a single value, JavaScript objects allow you to store collections of data and more complex entities. Think of an object like a physical noun, such as a

"Car." A car has properties (color, weight, brand) and methods (start, stop, brake). In JavaScript, you can define your own Custom Objects to model real-world data seamlessly.

To manipulate basic data and perform complex mathematical or date-related operations, JavaScript provides **Own (Built-in) Objects**. These are globally available objects that developers can use without creating them from scratch.

Common Built-in Objects include:

- **Math Object:** Allows you to perform mathematical tasks (e.g., `Math.random()`, `Math.round()`, `Math.PI`).
- **Date Object:** Used to work with dates and times (e.g., `new Date()`, `getFullYear()`).
- **String Object:** Provides methods to manipulate text (e.g., `toUpperCase()`, `length`, `substring()`).
- **Array Object:** Used to store multiple values in a single variable (e.g., `push()`, `pop()`, `length`).

Example: Custom Object & Built-in Object

```
// 1. Creating a Custom Object
let student = {
    firstName: "Rahul",
    lastName: "Sharma",
    age: 21,
    fullName: function() {
        return this.firstName + " " + this.lastName;
    }
};
console.log(student.fullName()); // Output: Rahul Sharma

// 2. Using Built-in Objects
let today = new Date();
console.log(today.getFullYear()); // Outputs the current year

let randomNum = Math.random(); // Outputs a random number between 0 and 1
```

Key Points:

- ✓ Objects are defined using curly braces `{}` (Object Literal notation).
- ✓ Properties are accessed using dot notation (`object.property`) or bracket notation (`object['property']`).
- ✓ The `this` keyword inside a method refers to the object that owns the method.

Hinglish Explanation:

*JavaScript mein sab kuch almost ek **Object** hai. Object ek container hota hai jisme hum variables (jise property kehte hain) aur functions (jise methods kehte hain) ko ek saath group kar sakte hain. Jaise 'Student' ek object hai jiska naam, roll number (property) aur padhai karna (method) ho sakta hai. Iske alawa JS mein pehle se bane huye objects hote hain jaise **Math** (calculations ke liye) aur **Date** (time ke liye), jinhe hum direct use kar sakte hain.*

2 DHTML, DOM, and Browser Environments

Definition:

DOM (Document Object Model) is a programming interface for web documents, representing the page so that programs can change the document structure, style, and content. **DHTML (Dynamic HTML)** is not a language, but a term used to describe the combination of HTML, CSS, and JavaScript to create interactive and animated websites. The **Browser Environment (BOM)** provides objects to interact with the browser window itself.

Detailed Explanation:

When a web page is loaded, the browser creates a Document Object Model of the page. The DOM is constructed as a tree of Objects. JavaScript uses the DOM to dynamically access and update the content, structure, and style of the document. Without the DOM, JavaScript wouldn't have any model or notion of web pages, HTML elements, or CSS styles.

DHTML is the practical application of the DOM. By combining HTML (for structure), CSS (for presentation), and JavaScript (for behavior via the DOM), developers can create dynamic content. This includes animations, drop-down menus, hiding/showing text, and changing colors on the fly based on user actions.

Events and Buttons: Interactivity in DHTML is driven by *Events*. An event is an action that occurs in the browser, such as a user clicking a button, hovering over an image, or pressing a key. JavaScript "listens" for these events (using event handlers like `onclick`, `onmouseover`, `onload`) and executes specific code when the event fires.

Controlling the Browser (BOM): The Browser Object Model allows JS to talk to the browser. The core object is the `window` object. It controls aspects outside the document itself, such as opening new tabs (`window.open`), showing alerts (`window.alert`), or getting screen width.

The DOM Tree Structure

```
                    [ Document ]
                          |
                    [ <html> ]
                   /          \
          [ <head> ]          [ <body> ]
             /                /         \
     [ <title> ]        [ <h1> ]     [ <a> ]
          |                 |            |
"Page Title"        "Heading"     "Click Here"
```

Example: DHTML & DOM Event Handling

```html
<!DOCTYPE html>
<html>
<body>
    <h2 id="header">Welcome to DOM</h2>

    <!-- Button with onclick event -->
    <button onclick="changeColor()">Change Color</button>

    <script>
```

```
        function changeColor() {
            // Accessing DOM and changing CSS
            document.getElementById("header").style.color = "red";

            // Controlling browser via BOM
            window.alert("Color changed successfully!");
        }
    </script>
</body>
</html>
```

Advantages of DHTML & DOM:

- ✓ Allows creation of highly interactive and engaging user interfaces.
- ✓ Updates parts of the page without requiring a full page reload from the server.
- ✓ Reduces server load since UI changes are processed on the client side.

Hinglish Explanation:

*Jab browser ek webpage load karta hai, toh wo uska ek tree-structure banata hai jise **DOM** kehte hain. **DHTML** koi nayi language nahi hai, balki HTML, CSS aur JS ka combination hai jisse website interactive banti hai. Jab aap kisi button par click karte hain (jisey **Event** kehte hain), toh JS DOM ka use karke page ka color, text ya design bina reload kiye change kar deta hai. Browser ke tab, alerts, aur screen size ko control karne ke liye **BOM (Browser Object Model)** ka use hota hai.*

3 Forms and Validations

Definition:

Form Validation is the process of checking user input data in an HTML form to ensure it meets specific requirements (e.g., correct format, not empty) before the data is submitted to the web server. It primarily relies on JavaScript to perform client-side validation.

Detailed Explanation:

Web forms are the primary way users send data to a server (e.g., registration, login, contact forms). However, users often make mistakes—typing text in a phone number field, forgetting the '@' in an email, or leaving mandatory fields blank. If this invalid data reaches the server, it can cause errors, crashes, or security vulnerabilities like SQL Injection.

To prevent this, developers use **Client-Side Validation** via JavaScript. By intercepting the form's submission event (`onsubmit`), JavaScript can check the values entered into the input fields. If the data is invalid, the script stops the form submission, prevents the page from reloading, and displays an error message immediately to the user. This saves time and reduces unnecessary traffic to the server.

Advanced validation often uses **Regular Expressions (Regex)**, which are patterns used to match

character combinations in strings. For example, a Regex pattern can precisely verify if a string follows the standard format of an email address (e.g., `text@domain.com`).

🔍 Key Validation Checks:

✓ **Presence check:** Ensuring required fields are not empty.

✓ **Format check:** Ensuring emails have '@' and phone numbers have digits (via Regex).

✓ **Length check:** Ensuring passwords are at least 8 characters long.

✓ **Matching check:** Ensuring "Password" and "Confirm Password" fields are identical.

💻 **Example: Simple Form Validation**

```html
<html>
<body>
    <!-- onsubmit returns true/false to allow/prevent submission -->
    <form name="myForm" onsubmit="return validateForm()" method="post">
        Name: <input type="text" id="fname" name="fname">
        <input type="submit" value="Submit">
    </form>

    <script>
        function validateForm() {
            // Get value of the input field
            let x = document.getElementById("fname").value;

            if (x == "") {
                alert("Name must be filled out");
                return false; // Prevents form from submitting
            }
            return true; // Allows submission
        }
    </script>
</body>
</html>
```

✅ **Advantages of Client-Side Validation:**

✓ Instant feedback to the user without waiting for a server response.

✓ Reduces unnecessary processing and bandwidth load on the web server.

❌ **Disadvantages:**

✗ Can be bypassed if the user disables JavaScript in their browser. (Therefore, *Server-Side validation is always mandatory* as a fallback).

👉 ***Hinglish Explanation:***

*Form **Validation** ka matlab hai check karna ki user ne form mein details sahi daali hain ya nahi (jaise email mein @ lagaya hai ya nahi, password khali to nahi chhod diya). Server par data bhejney se pehle, JS browser par hi ye check kar leta hai. Agar galti milti hai, toh JS form submit hone se rok deta hai aur ek*

error dikhata hai. Isse website fast chalti hai aur server par faltu ka load nahi padta.

4 AJAX (Asynchronous JavaScript and XML)

📌 Definition:

AJAX stands for Asynchronous JavaScript and XML. It is a set of web development techniques that allows web applications to send and retrieve data from a server asynchronously (in the background) without interfering with the display and behavior of the existing page.

📖 Detailed Explanation:

Before AJAX, if a user wanted to update a small part of a webpage (like liking a post or submitting a comment), the browser had to send a request to the server, and the server would return an entirely new HTML page. This full-page reload was slow, disruptive, and consumed a lot of bandwidth.

AJAX revolutionized web development. Using the `XMLHttpRequest` object (or the modern `Fetch API`), JavaScript can make an HTTP request to the server in the background. While the server processes the request, the user can continue interacting with the page seamlessly (hence, "Asynchronous"). When the server responds with data (traditionally XML, but nowadays almost always JSON), JavaScript receives it and uses the DOM to update only that specific part of the webpage.

AJAX Alternatives: While the traditional `XMLHttpRequest` is the foundation of AJAX, modern development relies on easier alternatives like the native **Fetch API**, third-party libraries like **Axios**, or **WebSockets** for real-time, two-way communication.

📊 Traditional vs AJAX Architecture

```
                    TRADITIONAL WEB APP:
Browser Request ----> Server Processing ----> Full HTML Page Reload

                    AJAX BASED WEB APP:
   Browser Event ----> JS XMLHttpRequest ----> Server Processing
            |                                          |
  (User keeps browsing)                   (Returns JSON/XML data)
            |                                          |
            +<------- JS updates DOM with new data <---+
```

✅ **Advantages of AJAX:**

✓ **No Page Reloads:** Updates parts of a page dynamically, creating a seamless app-like experience.

✓ **Reduced Bandwidth:** Transfers only the required data (JSON/XML) instead of entire HTML files.

✓ **Asynchronous:** Users are not blocked while waiting for server responses.

✗ **Disadvantages:**

✗ **SEO Difficulties:** Search engines historically struggle to index dynamically generated AJAX content.

✗ **Browser History:** The browser's "Back" button may not work as expected because the URL doesn't change during AJAX calls.

✗ Requires JavaScript to be enabled on the client browser.

🌍 **Applications of AJAX:**

- **Google Maps:** Dragging the map loads new map tiles in the background without reloading the page.
- **Social Media:** Infinite scrolling (loading new posts as you scroll down) and "Like" buttons.
- **Live Search:** Auto-complete suggestions appearing as you type in a search box.

👉 ***Hinglish Explanation:***

AJAX *ek technique hai jisse hum background mein server se data mangwa sakte hain, bina pura page refresh kiye. Jaise jab aap Instagram par post like karte hain, toh sirf like ka counter update hota hai, poora page dobara load nahi hota. Ye kaam* `XMLHttpRequest` *object ke through hota hai. Iske modern alternatives* ***Fetch API*** *aur* ***Axios*** *hain jo use karne mein zyada aasan hain.*

5 XML, Key Components, DTD and Schemas

📌 Definition:

XML (eXtensible Markup Language) is a flexible, text-based markup language designed to store, transport, and structure data. Unlike HTML, which defines how data should look, XML defines what the data *is* by allowing users to create their own customized tags.

📖 Detailed Explanation:

In a world where different systems (e.g., a Java application and a Python application) need to share data, they require a common, universally understood format. XML provides this standard. Because XML is plain text, it is platform-independent and human-readable. It structures data hierarchically using tags.

XML Key Components:

- **Elements:** The main building blocks, defined by opening and closing tags (e.g., `<student>...</student>`).
- **Attributes:** Additional information added to an element's opening tag (e.g., `<student id="101">`).
- **PCDATA (Parsed Character Data):** The actual text data between the tags that will be parsed by an XML parser.

Validating XML (DTD vs Schema): Anyone can make up tags in XML. However, if two companies are exchanging an XML invoice, they must agree on the exact structure (e.g., an invoice must have a `<date>` and `<amount>`). We define these rules using DTD or XML Schema.

1. DTD (Document Type Definition): The older way to define the structure, elements, and attributes of an XML document. It checks if the XML is "Valid". However, DTD has its own complex syntax and does not support data types (you can't enforce that an age must be a number).

2. XML Schema (XSD): The modern, powerful alternative to DTD. XML Schemas are written in XML itself. They support strict data types (string, integer, date), complex structures, and namespaces, making data validation much more precise.

Example: Simple XML Document

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- Root Element -->
<bookstore>
    <book category="programming"> <!-- Attribute -->
        <title>Learn JavaScript</title> <!-- Element -->
        <author>John Doe</author>
        <price>29.99</price>
    </book>
</bookstore>
```

Advantages / Uses of XML:

✓ Platform and language independent data transport.

✓ Highly extensible; you can define your own tags.

✓ Self-describing and human-readable structure.

Hinglish Explanation:

*HTML batata hai ki data kaisa dikhega, lekin **XML** batata hai ki data kya hai. XML mein hum apne khud ke tags bana sakte hain jaise `<name>` ya `<price>`. Ye alag-alag systems (jaise Java aur PHP) ke beech data share karne ke kaam aata hai. Koi galat tag use na kare, iske rules set karne ke liye hum **DTD** ya **XML Schema** ka use karte hain. Schema better hai kyunki wo data types (number, text) ko bhi check karta hai.*

6 XSL, XSLT, and XML Applications

Definition:

XSL (eXtensible Stylesheet Language) is a styling language for XML. **XSLT (XSL Transformations)** is a component of XSL used to transform XML documents into other formats like HTML, plain text, or another XML structure, so they can be displayed by a browser.

Detailed Explanation:

XML stores raw data. If you open an XML file in a browser, it just looks like a tree of text and tags; it is not formatted or visually appealing. To display XML data beautifully on a web page, it must be converted into HTML. This is the job of XSL and XSLT.

XSLT acts as a bridge. An XSLT stylesheet contains template rules. It uses a language called **XPath** to navigate through the XML tree, find specific elements (like "all books with price > 20"), and then

transforms that data into a specific HTML output format (like an HTML table).

XML with Applications: Beyond just displaying data, XML is deeply embedded in software applications. It is used as configuration files (like `pom.xml` in Maven or `web.xml` in Java), for storing complex datasets, drawing vector graphics (SVG is XML-based), and is the foundation for Web Services (SOAP).

📊 The XSLT Transformation Process

```
[ Raw XML Data ] + [ XSLT Stylesheet ]
                \                  /
                 \                /
                [ XSLT PROCESSOR ]
                          |
                          v
[ Formatted Output Document (HTML) ]
                          |
        (Displayed in Web Browser)
```

🔑 Key Features of XSLT:

- ✓ Transforms one XML document into another XML, HTML, or text document.
- ✓ Can add, remove, or rearrange elements and attributes.
- ✓ Allows data sorting and filtering before display.
- ✓ Uses `<xsl:template>` to define rules and `<xsl:for-each>` to loop through XML data.

Hinglish Explanation:

*XML sirf raw data hota hai, wo dikhne mein achha nahi lagta. Us raw data ko ek sundar HTML table ya design mein convert karne ke liye hum **XSLT** ka use karte hain. XSLT ek filter ya converter ki tarah kaam karta hai jo XML tags ko nikal kar unhe HTML tags mein badal deta hai taaki browser use acche se dikha sake.*

7 Introduction to Web Services

Definition:

A **Web Service** is a standardized software system or medium designed to support interoperable machine-to-machine interaction over a network. It allows applications built on different technologies or languages to communicate and share data over the internet using standard protocols like HTTP and XML/JSON.

Detailed Explanation:

In modern software, different applications need to talk to each other. For example, a flight booking website

(built in PHP) needs to check flight availability from an airline's database (built in Java). Because PHP and Java are different languages, they cannot communicate directly. **Web Services** solve this problem by acting as a universal translator.

The application requesting the data is the **Service Requestor (Client)**, and the application providing the data is the **Service Provider**. They communicate by sending standard text-based messages (XML or JSON) over HTTP. Because HTTP and XML/JSON are understood by every programming language, true *Interoperability* is achieved.

There are two primary types of Web Services:

1. **SOAP (Simple Object Access Protocol):** A strict, XML-based protocol. It uses **WSDL** (Web Services Description Language) to describe the service's functions, and **UDDI** (Universal Description, Discovery, and Integration) to list and discover available services. Highly secure, used in banking and enterprise apps.
2. **REST (Representational State Transfer):** An architectural style rather than a strict protocol. It is lightweight, uses standard HTTP methods (GET, POST, PUT, DELETE), and typically exchanges data in JSON format. It is the dominant method for modern web APIs (like Twitter or Google Maps API).

📊 Web Service Architecture (SOAP Model)

```
[ Service Requestor ] (e.g., PHP App)
                |        ^
                |        |
(Finds WSDL)  (Sends Data via XML/SOAP)
                v        |
[ Service Registry ] (UDDI) <---- [ Service Provider ] (e.g., Java App)
```

✅ **Advantages of Web Services:**

✓ **Interoperability:** Connects totally different platforms and languages.

✓ **Reusability:** A single web service (like a weather API) can be used by thousands of different mobile and web apps.

✓ **Standardized:** Uses well-known protocols (HTTP) so it easily passes through corporate firewalls.

👉 ***Hinglish Explanation:***

Web Services *internet par alag-alag softwares ko aapas mein baat karne ka ek standardized tareeka hai. Agar ek app Python mein bani hai aur doosri Java mein, toh wo dono direct baat nahi kar sakti. Par wo Web Services (SOAP ya REST) ka use karke ek doosre ko XML ya JSON format mein data bhej aur le sakti hain. Isi mechanism se saari modern APIs (jaise Payment gateways, Google login) kaam karti hain.*

🔁 QUICK REVISION NOTES

1. JS Objects:

JS relies on objects (key-value pairs). Custom objects represent real-world entities. Built-in objects (Math, Date, String) provide ready-to-use methods for calculations and data manipulation.

2. DOM & DHTML:
DOM is the tree-structure of HTML. DHTML is HTML+CSS+JS working together via Events (onclick) to change the DOM dynamically. BOM controls the browser window.

3. Forms Validations:
Client-side JS checks form inputs (empty fields, regex for email) before sending to the server. Saves server time and provides instant user feedback.

4. AJAX:
Asynchronous technique using `XMLHttpRequest`. Fetches data in the background and updates parts of a page without a full reload. Alternatives include Fetch API and Axios.

5. XML, DTD, Schemas:
XML stores/transports data using custom tags. DTD and XML Schema define the rules (structure) of the XML. Schema is better as it supports data types.

6. XSL & XSLT:
XSLT transforms raw, unformatted XML data into presentable HTML formats so browsers can render it nicely, using XPath to navigate the XML tree.

7. Web Services:
Allows interoperable machine-to-machine communication over HTTP. Major types: SOAP (strict, XML, uses WSDL) and REST (lightweight, uses JSON, standard HTTP methods).

📝 IMPORTANT QUESTIONS (6 Marks Each)

Q1. What is DHTML? Explain the relationship between DOM, HTML, CSS, and JavaScript in creating dynamic web pages. 6 Marks

Expected Answer Points:

- Define DHTML (combination, not a standalone language).
- Explain HTML's role (structure) and CSS's role (presentation).
- Explain the DOM (tree representation of HTML).
- Explain JS role: Intercepts events and uses the DOM API to modify HTML/CSS dynamically without reloading the page.

Q2. Why is form validation necessary? Explain how JavaScript is used for client-side validation with a brief example. 6 Marks

Expected Answer Points:

- Necessity: Prevents bad data (empty fields, bad formats) from hitting the server, preventing errors and SQL injections.
- Advantages of client-side: Instant feedback, saves server bandwidth.
- Provide a basic code snippet using `onsubmit` returning true/false.
- Mention Regex for format checking (like email).

Q3. Describe AJAX. What are its advantages and disadvantages? List some modern alternatives to traditional AJAX. 6 Marks

Expected Answer Points:

- Definition: Asynchronous JS and XML. Background data fetching using XMLHttpRequest.
- Advantages: No full page reload, highly interactive, saves bandwidth.
- Disadvantages: SEO issues, back-button tracking issues, requires JS enabled.
- Alternatives: Fetch API, Axios, WebSockets.

Q4. What is XML? Differentiate between DTD and XML Schema for XML validation. 6 Marks

Expected Answer Points:

- XML Definition: Markup language to store/transport data using custom tags. Focus on what data is, not how it looks.
- DTD (Document Type Definition): Older, uses own syntax, checks document structure but does NOT support data types.
- XML Schema (XSD): Newer, written in XML itself, highly extensible, strictly supports data types (string, int, date). Considered superior.

Q5. What is XSLT? Explain the process of transforming an XML document into an HTML document. 6 Marks

Expected Answer Points:

- Definition: XSL Transformations. Used to style/format raw XML data.
- Process: Takes XML data and an XSLT stylesheet as inputs into an XSLT Processor.
- Role of XPath: Navigates the XML tree to find specific elements.
- Outputs a transformed document, typically HTML, applying tags like tables and headers so the browser can render it beautifully.

Q6. Define Web Services. How do they promote interoperability among different software applications? Briefly compare SOAP and REST. 6 Marks

Expected Answer Points:

- Definition: Standardized machine-to-machine communication over a network.

- Interoperability: Explain how a Java app and PHP app can communicate because they send standard XML/JSON over universally accepted HTTP.
- SOAP: Strict protocol, relies solely on XML, highly secure, uses WSDL.
- REST: Architectural style, uses HTTP methods (GET/POST), usually exchanges lightweight JSON data.

These notes are comprehensive and exam-ready

Understand the core differences between DOM, AJAX, and XML concepts to score top marks!

Web Technologies - Unit 4 Notes (Adv. Scripting & XML)

Source: Rgpvnotes.tech | Premium Academic Content for Students

© 2026 - Your Academic Excellence Partner

These notes are created for educational purposes to help students master Web Technologies.